



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
RAI TECHNOLOGY UNIVERSITY

Estd. under Karnataka Act. No. 40 of 2013

Assignment No: 02

Subject Title: Deep Learning

Subject Code: 0CS351T

Submitted By:

Student Name: CHATTU UDAY KIRAN

University Enrolment Number: RTU24101CS009

Program Name: B. TECH CSE AIML

Semester: Vth Sem

Session: 2026-2027

Submitted To:

Faculty Name: Mr Kumar Satyam Tanti

College Of Engineering & Computer Application

Rai Technology University

Aim:-

To develop a simple neural network using TensorFlow/Keras to classify handwritten digits from 0 to 9 using the MNIST handwritten digit dataset, evaluate its performance, visualize the training process, and compare the results after performing one modification to the neural network.

Objective:-

1. To load and explore the MNIST dataset.
2. To build and train a neural network using TensorFlow/Keras.
3. To evaluate the model using test accuracy.
4. To visualize accuracy and loss during training.
5. To predict handwritten digits and compare actual and predicted labels.
6. To perform one experiment by modifying the model and compare the results.

1. Load and explore the MNIST dataset and display sample images.

- The MNIST dataset can be loaded directly using TensorFlow/Keras. The dataset is divided into training and testing sets.
- After loading the dataset, its dimensions and sample images are explored.

Code:-

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf.

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense, Dropout.
from PIL import Image, ImageOps
from google.colab import files
import io.
```

```
Print ("Tensorflow version:", tf._version_)
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.
datasets.mnist.load_data()
```

```
Print ("Training images:", x_train.shape)
```

```
Print ("Training labels:", y_train.shape)
```

```
Print ("Testing images:", x_test.shape)
```

```
Print ("Testing labels:", y_test.shape)
```

```
Print ("number of classes:", len(np.unique(y_train)))
```

```
Print ("classes:", np.unique(y_train))
```

```
Print ("Image size:", x_train[0].shape)
```

```
Plt.figure(figsize=(10,5))
```

```
for i in range(10):
```

```
    Plt.subplot(2,5,i+1)
```

```
    Plt.imshow(x_train[i], cmap="gray")
```

```
    Plt.title("label: " + str(y_train[i]))
```

```
    Plt.axis("off")
```

```
Plt.tight_layout()
```

```
Plt.show()
```

```
x_train = x_train.astype("float32")/255.0
```

```
x_test = x_test.astype("float32")/255.0
```

```
Print("Minimum pixel value:", x_train.min())
```

```
Print("Maximum pixel value:", x_train.max())
```

2. Design, compile, and train a neural network.

```
model = Sequential([
```

```
    Input(shape=(28,28)),
```

```
    Flatten(),
```

```
    Dense(128, activation='relu'),
```

```
    Dropout(0.2),
```

```
    Dense(10, activation='softmax')
```

```
])
```

```
model.compile(  
    optimizer = "adam",  
    loss = "sparse_categorical_crossentropy",  
    metrics = ["accuracy"]  
)
```

```
print("Model compiled successfully.")
```

```
history = model.fit(  
    x_train,  
    y_train,  
    epochs = 10,  
    batch_size = 32,  
    validation_split = 0.1  
)
```

```
test_loss, test_accuracy = model.evaluate(  
    x_test,  
    y_test,  
    verbose = 0  
)
```

```
print("Test Loss: ", test_loss)
```

```
print("Test Accuracy: ", test_accuracy)
```

```
print("Test Accuracy percentage: ", test_accuracy * 100)
```

Visualize training / validation accuracy and loss

```
plt.figure(figsize = (8, 5))
```

```
plt.plot(  
    history.history["accuracy"],  
    label = "Training Accuracy"  
)
```

```
plt.plot(  
    history.history["val-accuracy"],  
    label = "validation accuracy",  
)
```

```
plt.title("Training and validation Accuracy")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Accuracy")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(  
    history.history["loss"],  
    label = "Training loss",  
)
```

```
plt.plot(  
    history.history["val-loss"],  
    label = "Validation Loss",  
)
```

```
plt.title("Training and validation loss")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("loss")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

Test the model on 5 handwritten images and compare the actual and predicted labels.

```
Predictions = model.predict(x_test, verbose = 0)
```

```
Predicted_labels = np.argmax(predictions, axis = 1)
```

```
Print ("Actual labels: ", y_test[:10])
```

```
Print ("predicted labels: ", predicted_labels[:10])
```

```
plt.figure(figsize = (10, 5))
```

```
for i in range(10):
```

```
    plt.subplot(2, 5, i + 1)
```

```
    plt.imshow(x_test[i], cmap = "gray")
```

```
    plt.title (
```

```
        "Actual : " + str(y_test[i]) +
```

```
        "In Predicted : " + str(predicted_labels[i])
```

```
)
```

```
    plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
def preprocess_image(file_data):
```

```
    img = Image.open(
```

```
        io.BytesIO(file_data)
```

```
    ).convert("L")
```

```
    img = ImageOps.invert(img)
```

```
    bbox = img.getbbox()
```

```
if bbox:
    img = img.crop(bbox)
    img.thumbnail((20,20))
    Canvas = Image.new("L", (28,28), 0)
    x = (28 - img.width) // 2
    y = (28 - img.height) // 2
    canvas.paste(img, (x, y))
    image_array = image_array.astype("float32") / 255.0
    return image_array
```

```
from google.colab import files
uploaded = files.upload()
```

```
results = []
```

```
for filename, file_data in uploaded.items():
```

```
    image = preprocess_image(file_data)
```

```
    prediction = model.predict(
        image.reshape(1,28,28),
        verbose = 0
```

```
)
```

```
    predicted_digit = np.argmax(prediction)
```

```
    actual_digit = int(filename.split(".")[0])
```

```
    results.append(
```

```
        (filename, actual_digit, predicted_digit)
```

```
)
```

```
Print ("Handwritten Image Results")
```

```
Print ("-----")
```

```
for filename, actual, predicted in results:
```

```
    result = "correct" if actual == predicted else "Wrong"
```

```
    Print (
```

```
        "Image :", filename,
```

```
        " | Actual :", actual,
```

```
        " | Predicted :", predicted,
```

```
        " |", result
```

```
    )
```

```
plt.figure(figsize=(12,4))
```

```
for i, (filename, actual, predicted) in enumerate  
    (results):
```

```
    image = preprocess_image(  
        uploaded[filename]  
    )
```

```
plt.subplot(1, len(results), i+1)
```

```
plt.imshow(image, cmap="gray")
```

```
plt.title (
```

```
    f"Actual : {actual} | Predicted : {predicted}"
```

```
)
```

```
plt.axis ("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

Perform one experiment by changing either the number of neurons, activation function, number of layers, epochs, or adding dropout, and briefly compare the results.

```
experiment_model = Sequential([  
    Input(shape=(28,28)),  
    Flatten(),  
    Dense(64, activation="relu"),  
    Dropout(0.2),  
    Dense(10, activation="softmax")  
])
```

```
experiment_model.summary()
```

```
experiment_model.compile(  
    optimizer="adam",  
    loss='sparse_categorical_crossentropy',  
    metrics=["accuracy"]  
)
```

```
experiment_history = experiment_model.fit(  
    x_train,  
    y_train,  
    epochs=10,  
    batch_size=32,  
    validation_split=0.1  
)
```

```
experiment_loss, experiment_accuracy = (  
    experiment_model.evaluate(  
        x_test,  
        y_test,
```

Verbose = 0

)
)

Print (

"Experimental Test Loss:",
experiment - loss

)

Print (

"Experimental Test Accuracy:",
experiment - accuracy

)

Print (

"Experimental Accuracy Percentage:",
experiment - accuracy * 100

)

Print ("In MODEL COMPARISON")

Print ("-----")

Print (

"Original Model - 128 neurons:",
round(test - accuracy * 100, 2),
"% "

)

Print (

"Experimental Model - 64 neurons:",
round(experiment - accuracy * 100, 2),
"% "

)

difference = (

experiment - accuracy - test - accuracy

)* 100

MNIST HANDWRITTEN DIGIT CLASSIFICATION USING TENSORFLOW/KERAS

AIM:-

To develop a simple neural network using TensorFlow/Keras to classify handwritten digits from 0 to 9 using the MNIST dataset.

Objectives:-

- To load and explore the MNIST dataset.
- To build and train a neural network using tensorflow/keras.
- To evaluate the model using test accuracy.
- To visualize training and validation accuracy and loss.
- To test five handwritten images and compare actual and predicted labels.
- To modify one model parameter and compare the results.

Introduction:-

Handwritten digit recognition is a common application of Deep learning and image classification. The objective is to identify handwritten digits and classify them into ten classes ranging from 0 to 9.

In this project, the MNIST handwritten digit dataset is used to train a neural network

The MNIST dataset contains 70,000 grayscale handwritten digit images, including 60,000 training images and 10,000 testing images. Each image has a size of 28×28 pixels. Sample images are displayed to explore the dataset. The pixel values are normalized from 0-255 to 0-1 before training.

Evaluation and Results:-

After training the model is evaluated using the test dataset.

Test accuracy = %

Training and validation accuracy and loss are plotted using Matplotlib to observe the model's learning performance. The trained model is also tested on five handwritten images.

Experiment:-

One parameter of the neural network is changed, such as the number of neurons, activation function, number of layers, epochs, or dropout.

Original Accuracy : %

Modified Accuracy : %

The results are compared to understand how changing the model affects its classification performance.

Conclusion :-

The assignment successfully demonstrates handwritten digit classification using TensorFlow/Keras and the MNIST dataset. The model was trained, evaluated, and tested on handwritten images. Accuracy and loss graphs helped analyze the training process, while the additional experiment demonstrated the effect of changing a neural network parameter.